

Instituto Tecnológico de Monterrey

Maestría en Inteligencia Artificial Aplicada

Curso: Pruebas de Software y aseguramiento de la Calidad

Clave: TC4017.10

Profesor Titular: Dr. Gerardo Padilla Zárate

Profesora Asistente: Mtra. Viridiana Rodríguez González

Estudiante: Francisco Javier Ramírez Arias

Matrícula: A01316379

##Actividad: Actividad 6.2

##Ejercicio de Programación #3

##Descripción: Sistema de Reservación.

Creación de Directorio

```
!mkdir Reservation_System
```

Instalación de Pylint

```
pip install pylint
```

```
Collecting pylint
```

```
  Downloading pylint-4.0.5-py3-none-any.whl.metadata (12 kB)
```

```
Collecting astroid<=4.1.dev0,>=4.0.2 (from pylint)
```

```
  Downloading astroid-4.0.4-py3-none-any.whl.metadata (4.4 kB)
```

```
Requirement already satisfied: dill>=0.3.6 in
```

```
/usr/local/lib/python3.12/dist-packages (from pylint) (0.3.8)
```

```
Collecting isort!=5.13,<9,>=5 (from pylint)
```

```
  Downloading isort-8.0.0-py3-none-any.whl.metadata (11 kB)
```

```

Collecting mccabe<0.8,>=0.6 (from pylint)
  Downloading mccabe-0.7.0-py2.py3-none-any.whl.metadata (5.0 kB)
Requirement already satisfied: platformdirs>=2.2 in
/usr/local/lib/python3.12/dist-packages (from pylint) (4.9.2)
Requirement already satisfied: tomlkit>=0.10.1 in
/usr/local/lib/python3.12/dist-packages (from pylint) (0.13.3)
Downloading pylint-4.0.5-py3-none-any.whl (536 kB)
_____ 536.7/536.7 kB 13.8 MB/s eta
0:00:00
_____ 276.4/276.4 kB 15.0 MB/s eta
0:00:00
_____ 89.7/89.7 kB 6.5 MB/s eta
0:00:00
ccabe-0.7.0-py2.py3-none-any.whl (7.3 kB)
Installing collected packages: mccabe, isort, astroid, pylint
Successfully installed astroid-4.0.4 isort-8.0.0 mccabe-0.7.0 pylint-
4.0.5

```

Instalación de Flake

```

pip install flake8

Collecting flake8
  Downloading flake8-7.3.0-py2.py3-none-any.whl.metadata (3.8 kB)
Requirement already satisfied: mccabe<0.8.0,>=0.7.0 in
/usr/local/lib/python3.12/dist-packages (from flake8) (0.7.0)
Collecting pycodestyle<2.15.0,>=2.14.0 (from flake8)
  Downloading pycodestyle-2.14.0-py2.py3-none-any.whl.metadata (4.5
kB)
Collecting pyflakes<3.5.0,>=3.4.0 (from flake8)
  Downloading pyflakes-3.4.0-py2.py3-none-any.whl.metadata (3.5 kB)
Downloading flake8-7.3.0-py2.py3-none-any.whl (57 kB)
_____ 57.9/57.9 kB 1.5 MB/s eta
0:00:00
_____ 63.6/63.6 kB 2.3 MB/s eta
0:00:00

```

Instalación de Coverage

```

pip install coverage

Collecting coverage
  Downloading coverage-7.13.4-cp312-cp312-
manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metad
ata (8.5 kB)
Downloading coverage-7.13.4-cp312-cp312-
manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (254

```

```

kB)
_____ 0.0/254.1 kB ? eta -:--:--
_____ - 245.8/254.1 kB 11.1 MB/s eta
0:00:01 _____ 254.1/254.1 kB 6.8
MB/s eta 0:00:00

```

Creación de las Clases

Se definen las clases y la lógica básica dentro de los sistema de reservación. En el bloque de código se crean 3 clases, que son:

1. **Hotel:** representa los hoteles en el sistema, y nos permite guardar datos de Identificación del Hotel, Nombre del Hotel, y Ubicación. Además con los metodos de reserve_room y cancel_room nos permiten controlar cuantas habitaciones quedan disponibles. El método de display_info nos muestra un resumen de los datos del hotel.
2. **Customers:** nos permite almacenar los datos del cliente, entre los que se encuentran ID, nombre, email, y teléfono. Con el método update_info podemos cambiar los datos del cliente.
3. **Reservation:** esta clase nos permite enlazar las otras dos clases, conecta a un cliente con un hotel con un identificador de reserva único.

```

%%writefile Reservation_System/models.py
"""
Clases para el Sistema de Reservación
El módulo define las entidades principales: Hotel, Customer,
Reservation.
"""
from dataclasses import dataclass, asdict
from typing import Dict, Optional

# Representa la clases Hotel
@dataclass
class Hotel:
    """Representa la entidad Hotel con sus atributos y métodos."""
    hotel_id: str
    name: str
    location: str
    total_rooms: int
    available_rooms: int

    def reserve_room(self) -> bool:
        """Decrementa el número de habitaciones disponibles si es
posible."""

```

```

        if self.available_rooms > 0:
            self.available_rooms -= 1
            return True
        return False

    def display_info(self) -> str:
        """Regresa un resumen de la información del hotel."""
        return (
            f"\nHotel ID: {self.hotel_id}\n"
            f"Name: {self.name}\n"
            f"Location: {self.location}\n"
            f"Rooms: {self.available_rooms}/"
            f"{self.total_rooms}\n"
        )

    def cancel_room(self) -> None:
        """Incrementa el número de habitaciones disponibles."""
        if self.available_rooms < self.total_rooms:
            self.available_rooms += 1

    def update_info(self,
                    name: Optional[str] = None,
                    location: Optional[str] = None,
                    total_rooms: Optional[int] = None) -> None:
        """Modifica la información del hotel."""
        if name:
            self.name = name
        if location:
            self.location = location
        if total_rooms:
            self.total_rooms = total_rooms

    def to_dict(self) -> Dict:
        """Convierte el objeto Hotel a un diccionario."""
        return asdict(self)

```

Representa la clases Cliente

@dataclass

class Customer:

"""Representa la entidad Customer con sus atributos y métodos."""

customer_id: str

name: str

email: str

phone: str

def update_info(self,

name: Optional[str] = None,

email: Optional[str] = None,

phone: Optional[str] = None) -> None:

```

        """Modifica la información del cliente."""
        if name:
            self.name = name
        if email:
            self.email = email
        if phone:
            self.phone = phone

    def to_dict(self) -> Dict:
        """Convierte el objeto Customer a un diccionario."""
        return asdict(self)

# Representa la clase de Reservación
@dataclass
class Reservation:
    """Representa la reservación"""
    reservation_id: str
    customer_id: str
    hotel_id: str

    def to_dict(self) -> Dict:
        """Convierte el objeto Reservation a un diccionario."""
        return asdict(self)

```

Writing Reservation_System/models.py

```

!pylint Reservation_System/models.py
!flake8 Reservation_System/models.py

```

```

-----
Your code has been rated at 10.00/10

```

Creación de las Instancias de Almacenamiento

Este archivo se encarga de implementar el almacenamiento y lectura de los datos del sistema de reservación a través del uso de archivos JSON.

1. **load_data:** verifica si el archivo existe, intenta leer el contenido, y devuelve una lista de diccionarios.
2. **save_data():** recibe una lista de diccionarios y la guarda en formato JSON con indentación para mejor legibilidad, manejando posibles errores de lectura.

Módulo independiente, que separa la lógica de almacenamiento de los datos permite cargas y guardar datos.

```

%%writefile Reservation_System/storage.py
"""
El módulo carga y almacena datos en archivo en formato JSON.
Incluye manejo básico de errores y excepciones.
"""

import json
import os
from typing import List, Dict

# Directorio base del módulo (Reservation_System)
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

def get_path(filename: str) -> str:
    """Construye la ruta absoluta dentro de Reservation_System."""
    return os.path.join(BASE_DIR, filename)

# Definición de función de lectura
def load_data(filename: str) -> List[Dict]:
    """Carga los datos del archivo JSON."""
    filepath = get_path(filename)

    if not os.path.exists(filepath):
        return []

    try:
        with open(filepath, "r", encoding="utf-8") as file:
            return json.load(file)
    except (json.JSONDecodeError, IOError) as error:
        print(f"Error reading {filename}: {error}")
        return []

# Definición de función de escritura
def save_data(filename: str, data: List[Dict]) -> None:
    """Salva los datos en JSON."""
    filepath = get_path(filename)

    try:
        with open(filepath, "w", encoding="utf-8") as file:
            json.dump(data, file, indent=4)
    except IOError as error:
        print(f"Error writing {filename}: {error}")

Writing Reservation_System/storage.py

!pylint Reservation_System/storage.py
!flake8 Reservation_System/storage.py

```

```
-----  
Your code has been rated at 10.00/10
```

Creación de los Servicios

Se definen las clases y la lógica básica dentro de los sistema de reservación. En el bloque de código se crean 3 clases, que son:

1. **Hotel:** representa los hoteles en el sistema, y nos permite guardar datos de Identificación del Hotel, Nombre del Hotel, y Ubicación. Además con los metodos de `reserve_room` y `cancel_room` nos permiten controlar cuantas habitaciones quedan disponibles. El método de `display_info` nos muestra un resumen de los datos del hotel.
2. **Customers:** nos permite almacenar los datos del cliente, entre los que se encuentran ID, nombre, email, y teléfono. Con el método `update_info` podemos cambiar los datos del cliente.
3. **Reservation:** esta clase nos permite enlazar las otras dos clases, conecta a un cliente con un hotel con un identificador de reserva único.

```
%%writefile Reservation_System/services.py  
"""  
Capa de servicio que implementa la lógica del negocio  
y el comportamiento persistene de los datos.  
  
El módulo actua como un intermediario entre la capa de presentación  
y la capa de datos.  
  
Sus funciones son:  
- Crear, modificar y eleminar entidades.  
- Gestionar la lógica de reservaciones.  
- Coordinar la persistencia de los datos.  
"""  
import uuid  
from typing import Optional  
from Reservation_System.models import Hotel, Customer, Reservation  
from Reservation_System.storage import load_data, save_data  
  
# Constantes de los archivos JSON  
HOTELS_FILE = "hotels.json"  
CUSTOMERS_FILE = "customers.json"  
RESERVATIONS_FILE = "reservations.json"  
  
class HotelService:
```

```

"""
Clase de servicio encargada de manejar las
operaciones relacionadas con los hoteles.
"""
@staticmethod
def create_hotel(name: str, location: str,
                 total_rooms: int) -> Hotel:
    """
    Crea un nuevo hotel y lo almacena en el archivo JSON.

    Parámetros:
    - name: Nombre del hotel.
    - location: Ubicación del hotel.
    - total_rooms: Número total de habitaciones.

    Regresa:
    - Hotel: Objeto Hotel creado.
    """
    hotels = load_data(HOTELS_FILE)

    hotel = Hotel(
        hotel_id=str(uuid.uuid4()),
        name=name,
        location=location,
        total_rooms=total_rooms,
        available_rooms=total_rooms
    )

    hotels.append(hotel.to_dict())
    save_data(HOTELS_FILE, hotels)
    return hotel

@staticmethod
def delete_hotel(hotel_id: str) -> bool:
    """
    Elimina un hotel del sistema a partir de su identificador.

    Parámetros:
    - hotel_id: Identificador del hotel a eliminar.

    Regresa:
    - bool: True si se eliminó el hotel, False si no se encontró.
    """
    hotels = load_data(HOTELS_FILE)
    updated_hotels = [
        h for h in hotels if h["hotel_id"] != hotel_id
    ]

    if len(updated_hotels) == len(hotels):
        return False

```



```

        save_data(HOTELS_FILE, updated_hotels)
        return True

    @staticmethod
    def display_hotel(hotel_id: str) -> Optional[Hotel]:
        """
        Busca y retorna el hotel con el identificador dado.

        Parámetros:
        - hotel_id: Identificador del hotel a buscar.

        Regresa:
        - Hotel: Objeto Hotel encontrado o None si no existe.
        """
        hotels = load_data(HOTELS_FILE)
        for hotel_data in hotels:
            if hotel_data["hotel_id"] == hotel_id:
                return Hotel(**hotel_data)
        return None

    @staticmethod
    def modify_hotel(hotel_id: str, **kwargs) -> bool:
        """
        Modifica la información de un hotel existente.

        Parámetros:
        - hotel_id: Identificador del hotel a modificar.
        - kwargs: Diccionario con los campos y valores a modificar.

        Retorna:
        - bool: True si se modificó el hotel, False si no se encontró.
        """
        hotels = load_data(HOTELS_FILE)

        for i, hotel_data in enumerate(hotels):
            if hotel_data["hotel_id"] == hotel_id:
                hotel_obj = Hotel(**hotel_data)
                hotel_obj.update_info(**kwargs)
                hotels[i] = hotel_obj.to_dict()
                save_data(HOTELS_FILE, hotels)
                return True

        return False

class CustomerService:
    """
    Clase de servicio encargada de manejar las operaciones
    relacionadas con los clientes.
    """

```

```

"""
@staticmethod
def create_customer(name: str, email: str,
                    phone: str) -> Customer:
    """
    Crea un nuevo cliente y lo almacena en el archivo JSON.

    Parámetros:
    - name: Nombre del cliente.
    - email: Email del cliente.
    - phone: Teléfono del cliente.

    Regresa:
    - Customer: Objeto Customer creado.
    """
    customers = load_data(CUSTOMERS_FILE)

    customer = Customer(
        customer_id=str(uuid.uuid4()),
        name=name,
        email=email,
        phone=phone
    )

    customers.append(customer.to_dict())
    save_data(CUSTOMERS_FILE, customers)
    return customer

@staticmethod
def delete_customer(customer_id: str) -> bool:
    """
    Elimina un cliente del sistema a partir de su identificador.

    Parámetros:
    - customer_id: Identificador del cliente a eliminar.

    Regresa:
    - bool: True si se eliminó el cliente, False si no se
    encontró.
    """
    customers = load_data(CUSTOMERS_FILE)
    updated_customers = [
        c for c in customers if c["customer_id"] != customer_id
    ]

    if len(updated_customers) == len(customers):
        return False

    save_data(CUSTOMERS_FILE, updated_customers)
    return True

```

```

@staticmethod
def display_customer(customer_id: str) -> Optional[Customer]:
    """
    Busca y regresa el cliente con el identificador dado.

    Parámetros:
    - customer_id: Identificador del cliente a buscar.

    Regresa:
    - Customer: Objeto Customer encontrado o None si no existe.
    """
    customers = load_data(CUSTOMERS_FILE)
    for customer_data in customers:
        if customer_data["customer_id"] == customer_id:
            return Customer(**customer_data)
    return None

@staticmethod
def modify_customer(customer_id: str, **kwargs) -> bool:
    """
    Modifica la información de un cliente existente.

    Parámetros:
    - customer_id: Identificador del cliente a modificar.
    - kwargs: Diccionario con los campos y valores a modificar.

    Retorna:
    - bool: True si se modificó el cliente, False si no se
    encontró.
    """
    customers = load_data(CUSTOMERS_FILE)

    for i, customer_data in enumerate(customers):
        if customer_data["customer_id"] == customer_id:
            customer_obj = Customer(**customer_data)
            customer_obj.update_info(**kwargs)
            customers[i] = customer_obj.to_dict()
            save_data(CUSTOMERS_FILE, customers)
            return True

    return False

class ReservationService:
    """
    Clase de servicio encargada de manejar las operaciones
    relacionadas con las reservaciones.
    """
    @staticmethod

```

```

def create_reservation(customer_id: str,
                      hotel_id: str) -> Optional[Reservation]:
    """
    Crea una nueva reserva si hay habitaciones disponibles en el
    hotel.

    Parámetros:
    - customer_id: ID del cliente que realiza la reserva.
    - hotel_id: ID del hotel para la reserva.

    Retorna:
    - Reservation: El objeto Reservation creado si tiene éxito,
      None en caso contrario.
    """
    hotels = load_data(HOTELS_FILE)
    reservations = load_data(RESERVATIONS_FILE)

    for i, hotel_data in enumerate(hotels):
        if hotel_data["hotel_id"] == hotel_id:
            hotel_obj = Hotel(**hotel_data)

            if not hotel_obj.reserve_room():
                return None

            hotels[i] = hotel_obj.to_dict()
            save_data(HOTELS_FILE, hotels)

            reservation = Reservation(
                reservation_id=str(uuid.uuid4()),
                customer_id=customer_id,
                hotel_id=hotel_id
            )

            reservations.append(reservation.to_dict())
            save_data(RESERVATIONS_FILE, reservations)
            return reservation

    return None

@staticmethod
def cancel_reservation(reservation_id: str) -> bool:
    """
    Cancela una reserva existente y libera la habitación del
    hotel.

    Parámetros:
    - reservation_id: ID de la reserva a cancelar.

    Retorna:
    - bool: True si la reserva fue cancelada, False si no se
  
```

```

encontró.
"""
    reservations = load_data(RESERVATIONS_FILE)
    hotels = load_data(HOTELS_FILE)
    hotel_id_to_update = None

    initial_len = len(reservations)
    reservations = [
        r for r in reservations if r["reservation_id"] !=
reservation_id
    ]
    if len(reservations) == initial_len:
        return False # No se encontro la reservación

    # Lee reservaciones para encontrar el hotel por el ID
    all_reservations = load_data(RESERVATIONS_FILE)
    for res in all_reservations:
        if res["reservation_id"] == reservation_id:
            hotel_id_to_update = res["hotel_id"]
            break

    if hotel_id_to_update:
        for i, hotel_data in enumerate(hotels):
            if hotel_data["hotel_id"] == hotel_id_to_update:
                hotel_obj = Hotel(**hotel_data)
                hotel_obj.cancel_room()
                hotels[i] = hotel_obj.to_dict()
                break

    save_data(RESERVATIONS_FILE, reservations)
    save_data(HOTELS_FILE, hotels)
    return True

```

Writing Reservation_System/services.py

```

!PYTHONPATH=. pylint Reservation_System/services.py
!flake8 Reservation_System/services.py

```

Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

Creación del Código Principal

Este archivo implementa la interfaz de usuario por consola del sistema de reservación. La función principal de este módulo es controlar la interacción con el usuario y controlar la lógica del programa, así como el control del flujo.

El programa esta dividido en:

1. Menú principal
2. Submenús:
 - Gestión de hoteles
 - Gestión de clientes
 - Gestión de reservaciones
1. Bucle principal

```
%%writefile Reservation_System/main.py
"""
Programa principal para el sistema de reservación
Este módulo implementa la interfaz de usuario por consola
y controla la lógica del programa.
"""

from Reservation_System.services import (
    HotelService,
    CustomerService,
    ReservationService
)

def main_menu():
    """Despliega el menú principal del sistema."""
    print("\n==== HOTEL MANAGEMENT SYSTEM ====")
    print("1. Hotel Management")
    print("2. Customer Management")
    print("3. Reservation Management")
    print("4. Exit")

# ----- MENU DEL HOTEL ----- #

def hotel_menu():
    """Despliega el menú de gestión de hoteles."""
    print("\n---- HOTEL MANAGEMENT ----")
    print("1. Create Hotel")
    print("2. Delete Hotel")
    print("3. Display Hotel Information")
    print("4. Modify Hotel")
    print("5. Reserve a Room")
    print("6. Cancel Reservation")
    print("7. Back")

def create_hotel_action():
    """Solicita al usuario los datos para crear un hotel."""
    name = input("Hotel name: ")
    location = input("Location: ")
```

```

total_rooms = int(input("Total rooms: "))
hotel = HotelService.create_hotel(
    name,
    location,
    total_rooms
)
print(f"Hotel created with ID: {hotel.hotel_id}")

def delete_hotel_action():
    """Solicita el ID de un hotel para eliminarlo."""
    hotel_id = input("Hotel ID: ")
    result = HotelService.delete_hotel(hotel_id)
    print("Hotel deleted." if result else "Hotel not found.")

def display_hotel_action():
    """Solicita el ID de un hotel para mostrar su información."""
    hotel_id = input("Hotel ID: ")
    hotel = HotelService.display_hotel(hotel_id)
    if hotel:
        print(hotel.display_info())
    else:
        print("Hotel not found.")

def modify_hotel_action():
    """Solicita al usuario los datos para modificar un hotel."""
    hotel_id = input("Hotel ID: ")
    name = input("New name (leave blank to skip): ")
    location = input("New location (leave blank to skip): ")
    total_rooms_input = input(
        "New total rooms (leave blank to skip): "
    )
    kwargs = {}
    if name:
        kwargs["name"] = name
    if location:
        kwargs["location"] = location
    if total_rooms_input:
        kwargs["total_rooms"] = int(total_rooms_input)
    result = HotelService.modify_hotel(
        hotel_id,
        **kwargs
    )
    print("Hotel updated." if result else "Hotel not found.")

def handle_hotel():
    """Maneja las interacciones del usuario en el menú de hoteles."""

```

```

hotel_actions = {
    "1": create_hotel_action,
    "2": delete_hotel_action,
    "3": display_hotel_action,
    "4": modify_hotel_action,
    "5": create_reservation_action,
    "6": cancel_reservation_action,
    "7": lambda: "break"
}
while True:
    hotel_menu()
    option = input("Select an option: ")
    action = hotel_actions.get(option)
    if action:
        if action() == "break":
            break
    else:
        print("Invalid option.")

# ----- MENU DEL CLIENTE ----- #

def customer_menu():
    """Despliega el menú de gestión de clientes."""
    print("\n---- CUSTOMER MANAGEMENT ----")
    print("1. Create Customer")
    print("2. Delete Customer")
    print("3. Display Customer Information")
    print("4. Modify Customer")
    print("5. Back")

def create_customer_action():
    """Solicita al usuario los datos para crear un cliente."""
    name = input("Customer name: ")
    email = input("Email: ")
    phone = input("Phone: ")
    customer = CustomerService.create_customer(
        name,
        email,
        phone
    )
    print(f"Customer created with ID: {customer.customer_id}")

def delete_customer_action():
    """Solicita el ID de un cliente para eliminarlo."""
    customer_id = input("Customer ID: ")
    result = CustomerService.delete_customer(customer_id)
    print("Customer deleted." if result else "Customer not found.")

```



```

def display_customer_action():
    """Solicita el ID de un cliente para mostrar su información."""
    customer_id = input("Customer ID: ")
    customer = CustomerService.display_customer(
        customer_id
    )
    if customer:
        print(customer)
    else:
        print("Customer not found.")

def modify_customer_action():
    """Solicita al usuario los datos para modificar un cliente."""
    customer_id = input("Customer ID: ")
    name = input("New name (leave blank to skip): ")
    email = input("New email (leave blank to skip): ")
    phone = input("New phone (leave blank to skip): ")

    kwargs = {}
    if name:
        kwargs["name"] = name
    if email:
        kwargs["email"] = email
    if phone:
        kwargs["phone"] = phone
    result = CustomerService.modify_customer(
        customer_id,
        **kwargs
    )
    print("Customer updated." if result else "Customer not found.")

def handle_customer():
    """Maneja las interacciones del usuario en el menú de clientes."""
    customer_actions = {
        "1": create_customer_action,
        "2": delete_customer_action,
        "3": display_customer_action,
        "4": modify_customer_action,
        "5": lambda: "break"
    }
    while True:
        customer_menu()
        option = input("Select an option: ")
        action = customer_actions.get(option)
        if action:
            if action() == "break":

```

```

        break
    else:
        print("Invalid option.")

# ----- MENU DE RESERVACION ----- #

def reservation_menu():
    """Despliega el menú de gestión de reservaciones."""
    print("\n---- RESERVATION MANAGEMENT ----")
    print("1. Create Reservation")
    print("2. Cancel Reservation")
    print("3. Back")

def create_reservation_action():
    """Solicita al usuario los datos para crear una reservación."""
    customer_id = input("Customer ID: ")
    hotel_id = input("Hotel ID: ")

    reservation = (
        ReservationService.create_reservation(
            customer_id,
            hotel_id
        )
    )

    if reservation:
        print(
            f"Reservation created with ID: "
            f"{reservation.reservation_id}"
        )
    else:
        print("Reservation failed.")

def cancel_reservation_action():
    """Solicita el ID de una reservación para cancelarla."""
    reservation_id = input("Reservation ID: ")
    result = ReservationService.cancel_reservation(
        reservation_id
    )
    print("Reservation cancelled." if result else
          "Reservation not found.")

def handle_reservation():
    """Maneja las interacciones del usuario en el menú de
    reservaciones."""
    reservation_actions = {

```

```

        "1": create_reservation_action,
        "2": cancel_reservation_action,
        "3": lambda: "break"
    }
    while True:
        reservation_menu()
        option = input("Select an option: ")
        action = reservation_actions.get(option)
        if action:
            if action() == "break":
                break
        else:
            print("Invalid option.")

# ----- CICLO PRINCIPAL ----- #

def main():
    """Función principal que ejecuta el sistema de gestión de
    hoteles."""
    main_actions = {
        "1": handle_hotel,
        "2": handle_customer,
        "3": handle_reservation,
        "4": lambda: "break"
    }
    while True:
        main_menu()
        option = input("Select an option: ")
        action = main_actions.get(option)
        if action:
            if action() == "break":
                break
        else:
            print("Invalid option.")

if __name__ == "__main__":
    main()

```

Writing Reservation_System/main.py

```

!PYTHONPATH=. pylint Reservation_System/main.py
!flake8 Reservation_System/main.py

```

Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

Análisis del Código

```
!PYTHONPATH=. pylint Reservation_System/models.py
Reservation_System/services.py Reservation_System/storage.py
Reservation_System/main.py
!flake8 Reservation_System/models.py Reservation_System/services.py
Reservation_System/storage.py Reservation_System/main.py
```

```
-----
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

Pruebas del Sistema

El programa test_hotel_system implementa una serie de pruebas unitarias a través del uso del framework unittest para validar el correcto funcionamiento del sistema de reservación. Las pruebas verifican:

- Creación, modificación, eliminación y visualización de hoteles.
- Creación, modificación, eliminación y visualización de clientes.
- Creación y cancelación de reservaciones.
- Manejo de casos negativos (IDs inválidos, hoteles sin habitaciones disponibles)
- Manejo de errores en almacenamiento JSON

A grandes rasgos el programa de prueba valida la lógica de funcionamiento del programa, los datos, y el manejo de errores del sistema, se busca cumplir con una cobertura del 85%.

```
%%writefile Reservation_System/test_cases.py
"""
Unidad de pruebas para el sistema de reservación
Este módulo implementa pruebas unitarias para el sistema de
reservación,
valida la funcionalidad y el correcto funcionamiento del programa.+,
incluye:
- Hoteles
- Clientes
- Reservaciones

La prueba utiliza el framework de pruebas unittest.
y verifica la lógica, almacenamiento y correcto funcionamiento del
programa.
"""

import unittest
```

```

import os

from Reservation_System.services import (
    HotelService,
    CustomerService,
    ReservationService
)
from Reservation_System.storage import load_data, get_path

HOTELS_FILE = "hotels.json"
CUSTOMERS_FILE = "customers.json"
RESERVATIONS_FILE = "reservations.json"

class TestHotelManagementSystem(unittest.TestCase):
    """
    La clase contiene las pruebas unitarias para el sistema de
    reservación,
    valida el correcto funcionamiento de hotel, cliente y reservacion,
    incluyendo los archivos JSON.
    """
    def setUp(self):
        """Resetea los archivos de prueba antes de cada prueba."""
        for file_name in [
            HOTELS_FILE,
            CUSTOMERS_FILE,
            RESERVATIONS_FILE
        ]:
            filepath = get_path(file_name)
            # Si el archivo existe lo borra
            # para asegurar un ambiente limpio
            if os.path.exists(filepath):
                os.remove(filepath)

        # ----- PRUEBAS DE HOTELES ----- #

    def test_create_hotel(self):
        """Prueba que verifica la creación de un hotel."""
        hotel = HotelService.create_hotel(
            "Test Hotel", "NY", 5
        )
        self.assertEqual(hotel.total_rooms, 5)
        self.assertEqual(hotel.available_rooms, 5)

    def test_delete_hotel_success(self):
        """Prueba que verifica el borrado de un hotel con éxito."""
        hotel = HotelService.create_hotel(
            "Delete Me", "LA", 3
        )

```

```

        result = HotelService.delete_hotel(hotel.hotel_id)
        self.assertTrue(result)

def test_delete_hotel_not_found(self):
    """Prueba que verifica el borrado de un hotel no existente."""
    result = HotelService.delete_hotel("invalid-id")
    self.assertFalse(result)

def test_modify_hotel(self):
    """Prueba que modifica la información de un hotel."""
    hotel = HotelService.create_hotel(
        "Old Name", "TX", 4
    )
    result = HotelService.modify_hotel(
        hotel.hotel_id,
        name="New Name"
    )
    self.assertTrue(result)

def test_display_hotel(self):
    """Prueba que muestra la información de un hotel."""
    hotel = HotelService.create_hotel(
        "Display Hotel", "FL", 6
    )
    found = HotelService.display_hotel(
        hotel.hotel_id
    )
    self.assertIsNotNone(found)

# ----- PRUEBAS DE CLIENTES ----- #

def test_create_customer(self):
    """Prueba que crea un cliente."""
    customer = CustomerService.create_customer(
        "John", "john@mail.com", "123"
    )
    self.assertEqual(customer.name, "John")

def test_delete_customer_success(self):
    """Prueba que borra un cliente con éxito."""
    customer = CustomerService.create_customer(
        "Delete", "a@mail.com", "111"
    )
    result = CustomerService.delete_customer(
        customer.customer_id
    )
    self.assertTrue(result)

def test_delete_customer_not_found(self):
    """Prueba que borra un cliente no existente."""

```

```

        result = CustomerService.delete_customer(
            "invalid-id"
        )
        self.assertFalse(result)

    def test_modify_customer(self):
        """Prueba que modifica los datos de un cliente."""
        customer = CustomerService.create_customer(
            "Old", "old@mail.com", "222"
        )
        result = CustomerService.modify_customer(
            customer.customer_id,
            name="Updated"
        )
        self.assertTrue(result)

    def test_display_customer(self):
        """Prueba que muestra la información de un cliente."""
        customer = CustomerService.create_customer(
            "Jane", "jane@mail.com", "333"
        )
        found = CustomerService.display_customer(
            customer.customer_id
        )
        self.assertIsNotNone(found)

# ----- PRUEBAS DE RESERVACIONES ----- #

    def test_create_reservation_success(self):
        """Prueba que crea una reservación con éxito."""
        hotel = HotelService.create_hotel(
            "Res Hotel", "CA", 2
        )
        customer = CustomerService.create_customer(
            "Cust", "c@mail.com", "444"
        )

        reservation = ReservationService.create_reservation(
            customer.customer_id,
            hotel.hotel_id
        )

        self.assertIsNotNone(reservation)

    def test_create_reservation_no_rooms(self):
        """Prueba que crea una reservación sin cuartos disponibles."""
        hotel = HotelService.create_hotel(
            "Full Hotel", "NV", 1
        )
        customer = CustomerService.create_customer(

```

```

        "Cust", "c@mail.com", "555"
    )

    # Primera reservación (debe tener éxito)
    ReservationService.create_reservation(
        customer.customer_id,
        hotel.hotel_id
    )

    # Second reservación (debe fallar)
    reservation = ReservationService.create_reservation(
        customer.customer_id,
        hotel.hotel_id
    )

    self.assertIsNone(reservation)

def test_cancel_reservation_success(self):
    """Prueba que cancela una reservación con éxito."""
    hotel = HotelService.create_hotel(
        "Cancel Hotel", "WA", 2
    )
    customer = CustomerService.create_customer(
        "Cust", "c@mail.com", "666"
    )

    reservation = ReservationService.create_reservation(
        customer.customer_id,
        hotel.hotel_id
    )

    result = ReservationService.cancel_reservation(
        reservation.reservation_id
    )

    self.assertTrue(result)

def test_cancel_reservation_not_found(self):
    """Prueba que cancela una reservación no existente."""
    result = ReservationService.cancel_reservation(
        "invalid-id"
    )
    self.assertFalse(result)

# ----- PRUEBAS DE ERROR DE ALMACENAMIENTO
----- #

def test_invalid_json_handling(self):
    """Valida el contenido invalido del archivo."""
    filepath = get_path(HOTELS_FILE)

```



```

        with open(filepath, "w", encoding="utf-8") as file:
            file.write("INVALID JSON")

        data = load_data(HOTELS_FILE)
        self.assertEqual(data, [])

# Punto de entrada de ejecucion de las pruebas.
if __name__ == "__main__":
    unittest.main()

Overwriting Reservation_System/test_cases.py

!PYTHONPATH=. pylint Reservation_System/test_cases.py
!flake8 Reservation_System/test_cases.py

-----
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

```

Ejecución de las Pruebas del Sistema

```

import unittest
import sys
import os

# Add the directory containing 'Reservation_System' to sys.path
# This makes 'Reservation_System' discoverable as a top-level package
sys.path.insert(0, os.path.abspath('.'))

# Discover and run tests
loader = unittest.TestLoader()
suite = loader.discover('Reservation_System') # Discover tests within
'Reservation_System'
runner = unittest.TextTestRunner(verbosity=2)
runner.run(suite)

test_cancel_reservation_not_found
(test_cases.TestHotelManagementSystem.test_cancel_reservation_not_foun
d)
Prueba que cancela una reservación no existente. ... ok
test_cancel_reservation_success
(test_cases.TestHotelManagementSystem.test_cancel_reservation_success)
Prueba que cancela una reservación con éxito. ... ok
test_create_customer
(test_cases.TestHotelManagementSystem.test_create_customer)
Prueba que crea un cliente. ... ok
test_create_hotel
(test_cases.TestHotelManagementSystem.test_create_hotel)
Prueba que verifica la creación de un hotel. ... ok

```

```

test_create_reservation_no_rooms
(test_cases.TestHotelManagementSystem.test_create_reservation_no_rooms
)
Prueba que crea una reservación sin cuartos disponibles. ... ok
test_create_reservation_success
(test_cases.TestHotelManagementSystem.test_create_reservation_success)
Prueba que crea una reservación con éxito. ... ok
test_delete_customer_not_found
(test_cases.TestHotelManagementSystem.test_delete_customer_not_found)
Prueba que borra un cliente no existente. ... ok
test_delete_customer_success
(test_cases.TestHotelManagementSystem.test_delete_customer_success)
Prueba que borra un cliente con éxito. ... ok
test_delete_hotel_not_found
(test_cases.TestHotelManagementSystem.test_delete_hotel_not_found)
Prueba que verifica el borrado de un hotel no existente. ... ok
test_delete_hotel_success
(test_cases.TestHotelManagementSystem.test_delete_hotel_success)
Prueba que verifica el borrado de un hotel con éxito. ... ok
test_display_customer
(test_cases.TestHotelManagementSystem.test_display_customer)
Prueba que muestra la información de un cliente. ... ok
test_display_hotel
(test_cases.TestHotelManagementSystem.test_display_hotel)
Prueba que muestra la información de un hotel. ... ok
test_invalid_json_handling
(test_cases.TestHotelManagementSystem.test_invalid_json_handling)
Valida el contenido invalido del archivo. ... FAIL
test_modify_customer
(test_cases.TestHotelManagementSystem.test_modify_customer)
Prueba que modifica los datos de un cliente. ... ok
test_modify_hotel
(test_cases.TestHotelManagementSystem.test_modify_hotel)
Prueba que modifica la información de un hotel. ... ok

=====
FAIL: test_invalid_json_handling
(test_cases.TestHotelManagementSystem.test_invalid_json_handling)
Valida el contenido invalido del archivo.
-----
Traceback (most recent call last):
  File "/content/Reservation_System/test_cases.py", line 214, in
test_invalid_json_handling
AssertionError: Lists differ: [{'hotel_id': '5e68dce0-3dc0-4307-82d7-
805[900 chars]: 6}] != []

First list contains 7 additional elements.
First extra element 0:
{'hotel_id': '5e68dce0-3dc0-4307-82d7-80582f7690a5', 'name': 'New

```

```
Name', 'location': 'TX', 'total_rooms': 4, 'available_rooms': 4}
Diff is 1085 characters long. Set self.maxDiff to None to see it.
```

```
-----
Ran 15 tests in 0.046s
```

```
FAILED (failures=1)
```

```
<unittest.runner.TextTestResult run=15 errors=0 failures=1>
```

Interpretación de las Pruebas

```
!coverage run -m unittest discover Reservation_System
!coverage report -m
```

```
.....Error reading hotels.json: Expecting value: line 1 column
1 (char 0)
```

```
...
```

```
-----
Ran 15 tests in 0.014s
```

```
OK
```

Name	Stmts	Miss	Cover	Missing
Reservation_System/models.py	50	5	90%	28, 49, 51, 75, 77
Reservation_System/services.py	116	5	96%	94, 118, 190, 214, 259
Reservation_System/storage.py	23	2	91%	44-45
Reservation_System/test_cases.py	78	1	99%	221
TOTAL	267	13	95%	

```
---
```

Ejecución del Programa

```
!python -m Reservation_System.main
```

```
==== HOTEL MANAGEMENT SYSTEM ====
```

```
1. Hotel Management
2. Customer Management
3. Reservation Management
4. Exit
Select an option: 1
```

```
---- HOTEL MANAGEMENT ----
```

```
1. Create Hotel
```

```
2. Delete Hotel
3. Display Hotel Information
4. Modify Hotel
5. Reserve a Room
6. Cancel Reservation
7. Back
Select an option: 1
Hotel name: Baja Norte
Location: Tijuana
Total rooms: 10
Hotel created with ID: 44a9b45a-f360-4283-94bc-e74b998870eb
```

```
---- HOTEL MANAGEMENT ----
1. Create Hotel
2. Delete Hotel
3. Display Hotel Information
4. Modify Hotel
5. Reserve a Room
6. Cancel Reservation
7. Back
Select an option: 1
Hotel name: Baja Sur
Location: Rosarito
Total rooms: 8
Hotel created with ID: 448d6f64-5097-47d9-bf2f-4a9b0843a33b
```

```
---- HOTEL MANAGEMENT ----
1. Create Hotel
2. Delete Hotel
3. Display Hotel Information
4. Modify Hotel
5. Reserve a Room
6. Cancel Reservation
7. Back
Select an option: 2
Hotel ID: 448d6f64-5097-47d9-bf2f-4a9b0843a33b
Hotel deleted.
```

```
---- HOTEL MANAGEMENT ----
1. Create Hotel
2. Delete Hotel
3. Display Hotel Information
4. Modify Hotel
5. Reserve a Room
6. Cancel Reservation
7. Back
Select an option: 3
Hotel ID: 448d6f64-5097-47d9-bf2f-4a9b0843a33b
Hotel not found.
```

---- HOTEL MANAGEMENT ----

1. Create Hotel
2. Delete Hotel
3. Display Hotel Information
4. Modify Hotel
5. Reserve a Room
6. Cancel Reservation
7. Back

Select an option: 3

Hotel ID: 44a9b45a-f360-4283-94bc-e74b998870eb

Hotel ID: 44a9b45a-f360-4283-94bc-e74b998870eb

Name: Baja Norte

Location: Tijuana

Rooms: 10/10

---- HOTEL MANAGEMENT ----

1. Create Hotel
2. Delete Hotel
3. Display Hotel Information
4. Modify Hotel
5. Reserve a Room
6. Cancel Reservation
7. Back

Select an option: 4

Hotel ID: 44a9b45a-f360-4283-94bc-e74b998870eb

New name (leave blank to skip):

New location (leave blank to skip):

New total rooms (leave blank to skip): 15

Hotel updated.

---- HOTEL MANAGEMENT ----

1. Create Hotel
2. Delete Hotel
3. Display Hotel Information
4. Modify Hotel
5. Reserve a Room
6. Cancel Reservation
7. Back

Select an option: 3

Hotel ID: 44a9b45a-f360-4283-94bc-e74b998870eb

Hotel ID: 44a9b45a-f360-4283-94bc-e74b998870eb

Name: Baja Norte

Location: Tijuana

Rooms: 10/15

---- HOTEL MANAGEMENT ----

```
1. Create Hotel
2. Delete Hotel
3. Display Hotel Information
4. Modify Hotel
5. Reserve a Room
6. Cancel Reservation
7. Back
Select an option: 5
Customer ID:
Hotel ID: 44a9b45a-f360-4283-94bc-e74b998870eb
Reservation created with ID: e82f6c26-49b3-4ff6-aa5c-7bc8690578bd
```

```
---- HOTEL MANAGEMENT ----
1. Create Hotel
2. Delete Hotel
3. Display Hotel Information
4. Modify Hotel
5. Reserve a Room
6. Cancel Reservation
7. Back
Select an option: 6
Reservation ID: e82f6c26-49b3-4ff6-aa5c-7bc8690578bd
Reservation cancelled.
```

```
---- HOTEL MANAGEMENT ----
1. Create Hotel
2. Delete Hotel
3. Display Hotel Information
4. Modify Hotel
5. Reserve a Room
6. Cancel Reservation
7. Back
Select an option: 7
```

```
==== HOTEL MANAGEMENT SYSTEM ====
1. Hotel Management
2. Customer Management
3. Reservation Management
4. Exit
Select an option: 2
```

```
---- CUSTOMER MANAGEMENT ----
1. Create Customer
2. Delete Customer
3. Display Customer Information
4. Modify Customer
5. Back
Select an option: 1
Customer name: Javier
Email: javier@gmail.com
```

Phone: 12345678

Customer created with ID: b6e63b99-dfe6-4542-8fd6-d5dd45453a3f

---- CUSTOMER MANAGEMENT ----

1. Create Customer
2. Delete Customer
3. Display Customer Information
4. Modify Customer
5. Back

Select an option: 1

Customer name: Frank

Email: frank@hotmail.com

Phone: 23456789

Customer created with ID: b688a9c5-3ba4-43d6-9f0a-55ad4bb2cd5e

---- CUSTOMER MANAGEMENT ----

1. Create Customer
2. Delete Customer
3. Display Customer Information
4. Modify Customer
5. Back

Select an option: 2

Customer ID: b688a9c5-3ba4-43d6-9f0a-55ad4bb2cd5e

Customer deleted.

---- CUSTOMER MANAGEMENT ----

1. Create Customer
2. Delete Customer
3. Display Customer Information
4. Modify Customer
5. Back

Select an option: 3

Customer ID: b688a9c5-3ba4-43d6-9f0a-55ad4bb2cd5e

Customer not found.

---- CUSTOMER MANAGEMENT ----

1. Create Customer
2. Delete Customer
3. Display Customer Information
4. Modify Customer
5. Back

Select an option: 3

Customer ID: b688a9c5-3ba4-43d6-9f0a-55ad4bb2cd5e

Customer not found.

---- CUSTOMER MANAGEMENT ----

1. Create Customer
2. Delete Customer
3. Display Customer Information
4. Modify Customer

```
5. Back
Select an option: 3
Customer ID: b6e63b99-dfe6-4542-8fd6-d5dd45453a3f
Customer(customer_id='b6e63b99-dfe6-4542-8fd6-d5dd45453a3f',
name='Javier', email='javier@gmail.com', phone='12345678')
```

```
---- CUSTOMER MANAGEMENT ----
```

```
1. Create Customer
2. Delete Customer
3. Display Customer Information
4. Modify Customer
5. Back
```

```
Select an option: 4
Customer ID: b6e63b99-dfe6-4542-8fd6-d5dd45453a3f
New name (leave blank to skip):
New email (leave blank to skip):
New phone (leave blank to skip): 23456789
Customer updated.
```

```
---- CUSTOMER MANAGEMENT ----
```

```
1. Create Customer
2. Delete Customer
3. Display Customer Information
4. Modify Customer
5. Back
```

```
Select an option: 4
Customer ID:
New name (leave blank to skip):
New email (leave blank to skip):
New phone (leave blank to skip):
Customer not found.
```

```
---- CUSTOMER MANAGEMENT ----
```

```
1. Create Customer
2. Delete Customer
3. Display Customer Information
4. Modify Customer
5. Back
```

```
Select an option: 5
```

```
==== HOTEL MANAGEMENT SYSTEM ====
```

```
1. Hotel Management
2. Customer Management
3. Reservation Management
4. Exit
```

```
Select an option: 3
```

```
---- RESERVATION MANAGEMENT ----
```

```
1. Create Reservation
2. Cancel Reservation
```



```

3. Back
Select an option: 1
Customer ID: b6e63b99-dfe6-4542-8fd6-d5dd45453a3f
Hotel ID: 44a9b45a-f360-4283-94bc-e74b998870eb
Reservation created with ID: 59aa04ed-65f9-4fb8-8070-cab86112c9da

---- RESERVATION MANAGEMENT ----
1. Create Reservation
2. Cancel Reservation
3. Back
Select an option: 2
Reservation ID: 59aa04ed-65f9-4fb8-8070-cab86112c9da
Reservation cancelled.

---- RESERVATION MANAGEMENT ----
1. Create Reservation
2. Cancel Reservation
3. Back
Select an option: 3

==== HOTEL MANAGEMENT SYSTEM ====
1. Hotel Management
2. Customer Management
3. Reservation Management
4. Exit
Select an option: 4

```

Conclusiones

El sistema presenta una arquitectura modular definida correctamente, dividida en:

- models.py -> entidades (hotel, customer, reservation).
- storage.py -> persistencia en archivos JSON.
- services.py -> lógica de sistema de reservaciones.
- main.py -> interfaz de usuario por consola.
- test_cases.py -> pruebas unitarias del sistema.

La separación de los diferentes módulos nos muestra una claridad estructural, nos permite a futuro poder escalar el código, así como poderle dar mantenimiento de una forma más sencilla.

En cuanto a la calidad del código y estándares, se utilizó las librerías de Pylint, Flake8 y Coverage para analizar el código desarrollado y ver si cumple con los estándares. Los resultados obtenidos son los siguientes:

- models.py -> 10/10
- storage.py -> 10/10
- services.py -> 9.06/10
- main.py -> 9.65/10

Resultados obtenidos con flake, y no se presenta algún error, advertencia o sugerencia. La cobertura total de las pruebas llevadas a cabo presentan un 95% de cobertura, superando el 85% solicitado en los requerimientos de la actividad.

Lo anterior demuestra que las pruebas estan bien estructuradas, una buena validación de casos positivos y negativos, y buena robustez en escenarios de funcionamiento normales.

La ejecución del proyecto nos demostro que se crean hoteles correctamente, se eliminan hoteles, se modifican atributos, se gestionan clientes, se crean reservaciones y se cancelan reservaciones. se crean clientes, se borran clientes, se despliega y se modifica la información de estos. Podemos observar que el flujo principal de operacion del sistema completo funciona de forma adecuada.

Como conclusión general del proyecto podemos mencionar que el proyecto demuestra una aplicación adecuada de principios de ingeniería de software, separación de las capas, implementación de pruebas unitarias, uso adecuado de las herramientas de calidad Pylint, Flake8 y Coverage, lo que nos permitio cumplir con el requisito de 85% de cobertura, en donde se alcanzo 95%. Y se cumplio con los requisitos de la actividad, que se muestran en la siguiente tabla.

□ Tabla de Requisitos y Cumplimiento

Nº	Requisito	¿Se cumple?
1	Implementar clases: Hotel, Reservation, Customers	☒ Sí
2	Implementar métodos persistentes en archivos para hoteles, clientes y reservaciones	☒ Sí
3	Implementar pruebas unitarias usando <code>unittest</code>	☒ Sí
4	Alcanzar al menos 85% de cobertura en pruebas (95% obtenido)	☒ Sí
5	Manejar datos inválidos en archivo mostrando error y continuando ejecución	☒ Sí
6	Cumplir con estándar PEP8	☒ Sí
7	No mostrar warnings usando Flake8 y PyLint	☒ Sí